

TUGAS INDIVIDU - ADT QUEUE

Muhammad Farhan Abdul Azis/24060124140166/Kelas E

11 Oktober 2025

Simulasi penjadwalan proses CPU dengan algoritma Round Robin menggunakan ADT Queue dan ADT Proses

1. Modul T_Proces

```
DEKLARASI/DEFINISI & SPESIFIKASI TIPE & PROTOTYPE
Type tProses = <idProses: character, burstTime: integer>
{tproses sebagai representasi proses yang akan dikerjakan di cpu}
```

createProses

```
DEKLARASI/DEFINISI & SPESIFIKASI TIPE & PROTOTYPE
procedure createProses(output P: tProses, input c: character, input
n: integer)
{I.S.: c dan n terdefinisi}
{F.S.: P terdefinisi dengan idProses = c dan burstTime = n}
{Proses: mengisi struktur proses dengan ID dan burst time yang
diberikan}
```

BODY/REALISASI PROTOTYPE

```
procedure createProses(output P: tProses, input c: character, n:
integer)
Kamus
    {tidak ada variabel lokal}
Algoritma
    P.idProses ← c
    P.burstTime ← n
```

idProses

```
DEKLARASI/DEFINISI & SPESIFIKASI TIPE & PROTOTYPE
function idProses(P: tProses) → character
{I.S.: P terdefinisi}
{F.S.: mengembalikan nilai idProses dari P}
{Proses: mengakses field idProses dari struktur tProses}
```

BODY/REALISASI PROTOTYPE
<pre> function idProses(P: tProses) → <u>character</u> Kamus {tidak ada variabel lokal} Algoritma → P.idProses </pre>

burstTime
DEKLARASI/DEFINISI & SPESIFIKASI TIPE & PROTOTYPE <pre> function burstTime(P: tProses) → <u>integer</u> {I.S.: P terdefinisi} {F.S.: mengembalikan nilai burstTime dari P} {Proses: mengakses field burstTime dari struktur tProses} </pre>
BODY/REALISASI PROTOTYPE <pre> function burstTime(P: tProses) → <u>integer</u> Kamus {tidak ada variabel lokal} Algoritma → P.burstTime </pre>

2. Modul T_Queue2

createQueue2
DEKLARASI/DEFINISI & SPESIFIKASI TIPE & PROTOTYPE <pre> procedure createQueue2(<u>output</u> Q: tQueue2) {I.S.: -} {F.S.: Q terdefinisi dan kosong} {Proses: menginisialisasi queue dengan semua elemen kosong ('\#'), head = 0, tail = 0} </pre>

enqueue2
DEKLARASI/DEFINISI & SPESIFIKASI TIPE & PROTOTYPE <pre> procedure enqueue2(<u>input/output</u> Q: tQueue2, <u>input</u> P: tProses) {I.S.: Q dan P terdefinisi} {F.S.: elemen queue Q bertambah 1 bila belum penuh} {proses: menambah elemen wadah Q, jika tail(Q)=kapasitas, maka semua elemen digeser lebih dulu sehingga head(Q)=1} </pre>

BODY/REALISASI PROTOTIPE

procedure enqueue2(input/output Q: tQueue2, input P: tProses)

Kamus

{tidak ada variabel lokal}

Algoritma

if isTailStop(Q) and (Q.head > 1) then

 resetHead(Q)

{endif}

if isEmptyQueue2(Q) then

 Q.head ← 1

 Q.tail ← 1

 Q.wadah[Q.tail] ← P

else

if not isFullQueue2(Q) then

 Q.tail ← Q.tail + 1

 Q.wadah[Q.tail] ← P

 [endif]

[endif]

dequeue2

DEKLARASI/DEFINISI & SPESIFIKASI TIPE & PROTOTIPE

procedure dequeue2(input/output Q: tqueue2, output P: tProses)

{I.S.: Q terdefinisi}

{F.S.: elemen head dikeluarkan dari queue dan disimpan di P}

{Proses:

- Jika queue tidak kosong, ambil elemen head
- Jika tinggal 1 elemen, set head = 0 dan tail = 0
- Selain itu, increment head
- Jika queue kosong, set P dengan proses '@'}

BODY/REALISASI PROTOTIPE

procedure dequeue2(input/output Q: tqueue2, output P: tProses)

Kamus

prosesKosong : tProses

Algoritma

prosesKosong.idProses ← '#'

prosesKosong.burstTime ← 0

if not isEmptyQueue2(Q) then

 P ← Q.wadah[Q.head]

```

    Q.wadah[Q.head] ← prosesKosong
    if isOneElement2(Q) then
        Q.head ← 0
        Q.tail ← 0
    else
        Q.head ← Q.head + 1
    [endif]
else
    P.idProses ← '@'
    P.burstTime ← 0
[endif]

```

3. Modul Utama

program_utama

DEKLARASI/DEFINISI & SPESIFIKASI TIPE & PROTOTYPE

```

program round_robin
{I.S.: -}
{F.S.: simulasi penjadwalan Round Robin selesai dan ditampilkan ke
layar}
{Proses:
    1. Membuat 5 proses berdasarkan 5 digit terakhir NIM (30069)
    2. Memasukkan semua proses ke queue
    3. Melakukan simulasi Round Robin dengan quantum time = 5
    4. Menampilkan urutan eksekusi, waktu mulai, dan waktu selesai}
{Algoritma Round Robin:
    - Ambil proses dari head queue
    - Eksekusi selama quantum time atau sampai selesai
    - Jika belum selesai, masukkan kembali ke tail queue
    - Ulangi sampai semua proses selesai}

```

BODY/REALISASI PROTOTYPE

Kamus

```

Use T_Queue2
Use T_Proces

```

Q: tqueue2	{antrian proses di CPU}
P1, P2, P3, P4, P5: tProses	{5 proses yang akan dijadwalkan}
currentProses: tProses	{proses yang sedang dieksekusi}
quantumTime: integer	{quantum time untuk Round Robin}
currentTime: integer	{waktu sistem saat ini}
processCount: integer	{counter untuk nomor eksekusi}
startTime: integer	{waktu mulai eksekusi proses}
executionTime: integer	{waktu eksekusi aktual}

Algoritma

```
quantumTime ← 5
currentTime ← 0
processCount ← 1

createProses(P1, 'A', 4)      {BT = 4}
createProses(P2, 'B', 6)      {BT = 0 → 4+3 = 7}
createProses(P3, 'C', 9)      {BT = 1}
createProses(P4, 'D', 6)      {BT = 6}
createProses(P5, 'E', 6)      {BT = 6}

createQueue2(Q)
enqueue2(Q, P1)
enqueue2(Q, P2)
enqueue2(Q, P3)
enqueue2(Q, P4)
enqueue2(Q, P5)

while not isEmptyQueue2(Q) do
    dequeue2(Q, currentProses)
    startTime ← currentTime    {Tentukan waktu mulai eksekusi}

    {Tentukan waktu eksekusi}
    if currentProses.burstTime ≤ quantumTime then
        executionTime ← currentProses.burstTime
        currentProses.burstTime ← 0
    else
        executionTime ← quantumTime
        currentProses.burstTime ← currentProses.burstTime -
quantumTime
    {endif}

    currentTime ← currentTime + executionTime
    processCount ← processCount + 1

    if currentProses.burstTime > 0 then
        enqueue2(Q, currentProses)
    {endif}
{endwhile}
```

Penjelasan Algoritma

1. Setiap proses diberi waktu maksimal untuk eksekusi

2. Proses yang belum selesai dikembalikan ke akhir antrian
3. Semua proses mendapat kesempatan yang sama

Langkah-langkah Round Robin:

1. Ambil proses dari head queue
2. Eksekusi proses selama quantum time atau sampai selesai (yang lebih kecil)
3. Jika proses belum selesai, masukkan kembali ke tail queue
4. Ulangi sampai semua proses selesai

```

● === SIMULASI PENJADWALAN PROSES CPU - ROUND ROBIN ===
NIM (5-digit terakhir): 40166
Quantum Time: 5 satuan waktu

=== PEMBENTUKAN PROSES BERDASARKAN NIM ===
Proses A: BT = 4
Proses B: BT = 7
Proses C: BT = 1
Proses D: BT = 6
Proses E: BT = 6

=== URUTAN KEDATANGAN PROSES ===
Proses A masuk antrian (BT: 4)
Proses B masuk antrian (BT: 7)
Proses C masuk antrian (BT: 1)
Proses D masuk antrian (BT: 6)
Proses E masuk antrian (BT: 6)

=== ANTRIAN PROSES AWAL ===
A B C D E

=== SIMULASI EKSEKUSI DENGAN ROUND ROBIN ===
Eksekusi | Proses | Waktu Mulai | Waktu Selesai | Burst Time Sisa
-----:|:-----:|:-----:|:-----:|:-----:
1 | A | 0 | 4 | 0
2 | B | 4 | 9 | 2
3 | C | 9 | 10 | 0
4 | D | 10 | 15 | 1
5 | E | 15 | 20 | 1
6 | B | 20 | 22 | 0
7 | D | 22 | 23 | 0
8 | E | 23 | 24 | 0

=== SIMULASI SELESAI ===
Total waktu eksekusi: 24 satuan waktu
Semua proses telah selesai dieksekusi.

```